

ERI / CLI

Design specification.

Eric Li

Infrastructure, systems engineering
and portfolio

DESIGN SYSTEM / INTERACTION / DELIVERY

A GUIDE TO THE SYSTEM

Contents

01	Project overview	3
02	Design direction and visual system	5
03	Pages, navigation and content placement	10
04	Interactive landscape	12
05	Responsive behavior and accessibility	16
06	Loading and rendering lifecycle	18
07	Editorial approach and publishing	20
08	Technical reference	22
09	Appendix: implementation excerpts	25

SCOPE AND VERIFICATION

Implementation details are based on the project specification and code excerpts, without an independent review of the repository or deployed site. Operational figures, the 46-article count and the approximate 604 KB script size describe that project snapshot. Transfer sizes and performance have not been measured for this document, and no accessibility audit is implied. Open implementation questions are noted where they affect the design or behavior.

SECTION 01

Project overview

An introduction to the work, the operational context and the portfolio's priorities.

01



SECTION 01

Project overview

ERI / CLI presents Eric Li's work with self-hosted homelabs, Linux clusters, containers and storage, alongside his development toward network engineering and site reliability. The portfolio gives visitors a concise introduction, selected project evidence and a route into longer technical writing.

The homepage leads with operational context: 29 users, 23 active users and 5,769 hours logged in the preceding year across UniFi, Proxmox and Docker. These are presented as portfolio content, with supporting project explanations, rather than as a live monitoring dashboard.

The design combines large editorial typography with a dark background and a procedural wireframe landscape. The landscape gives the site a recognizable visual identity; the text explains what Eric operates, how the systems work and what he has learned from maintaining them.

The main priorities are to:

- Explain the work through named systems, project details and concrete figures.
- Give headlines and technical prose a clear reading order.
- Use the landscape as an authored visual element, without presenting decoration as live telemetry.
- Make room for unfinished notes, incident reviews and operational mistakes when they help explain the work.

SECTION 02

Design direction and visual system

Editorial Brutalism, expressed through typography,
color and an asymmetric desktop grid.

02



SECTION 02

Design direction and visual system

The visual direction is *Editorial Brutalism*: tightly set serif headlines, an uneven three-column grid and a wireframe landscape whose individual pillars remain visible. The large type gives the opening statement room to speak; smaller labels keep operational figures and project facts easy to scan.

The $3 \times 3 \times 3 \times 3$ concept is a design constraint: three type families, three core colors, three homepage sections and three desktop columns. It provides consistency without requiring every component or explanation to repeat the same pattern.

Typography

Each family has a defined role:

FAMILY	USE	TREATMENT
PP Kyoto	Hero headline, major section headings and project titles	Display serif; line height 0.78–0.85, tracking -0.055em–-0.065em
AUTHENTIC Sans Pro	Body copy, project descriptions, article prose and primary navigation	The main reading face; registered as 'AUTHENTIC Sans' in the CSS token
Pitch Sans	Metadata, timestamps, hardware statistics, categories and technical tags	Monospaced text, uppercase tracking where appropriate and font-variant-numeric: tabular-nums

PP Kyoto's tight line spacing makes a multiline headline read as one compact shape. Longer explanations move into AUTHENTIC Sans. Pitch Sans handles details such as 2026-08-10 • 3 MIN READ, giving dates and measurements a distinct place on the page.

DESIGN DIRECTION AND VISUAL SYSTEM / CONTINUED

Color

Black and white carry most of the interface. Blue marks selected interactions and higher terrain elevations.

COLOR	VALUE	USE
Void Black	#08090C	Main background, --ink
Dark background variant	#0A0B0E	Supporting background tone, --ink-mass
Titan White	#E5E7EB	Text, structural lines and low terrain elevations
International Klein Blue	#002FA7	Accent token, terrain summits, active link underlines and cursor highlights

Titan White uses four opacity levels to distinguish information:

- **100%:** primary headlines and active states.
- **72%:** body prose and secondary descriptions, through --signal-72.
- **48%:** metadata, hardware tags and captions, through --signal-48.
- **24%:** subtle gridlines, through --signal-24.

Blue is reserved for focal details rather than broad background fills. Active links use `text-decoration-color: var(--accent)`. Terrain brightness is handled separately in the elevation shader (Section 4).

DESIGN DIRECTION AND VISUAL SYSTEM / CONTINUED

Desktop layout

The desktop grid uses a nominal **23 : 49 : 28** column ratio. The wider center gives headlines and featured projects more room; the side columns hold statistics, context and supporting facts.

COLUMN	HOMEPAGE INTRODUCTION	SELECTED PROJECTS
Left, 23 parts	Operational statistics	Project selector
Center, 49 parts	Main headline	Featured project card
Right, 28 parts	Background and explanation	Project facts

The grid uses fractional tracks with minimum widths of 190px, 410px and 240px. The ratio therefore describes the intended distribution, not an exact percentage of the full viewport at every width. Gaps and minimum track sizes also affect the result. See Appendix A for the grid CSS.

THE DESKTOP GRID

An uneven grid. A clear reading order.



Operational statistics

Main headline

Background and explanation

Figure 01. The live introduction at 1600 × 1000 CSS pixels. Guides identify the content-column boundaries. The 23 : 49 : 28 labels describe nominal track proportions, not percentages of the viewport; minimum widths and gaps affect the rendered layout.

Implementation capture · 12 September 2026 · [ERI / CLI](#)

SECTION 03

Pages, navigation and content placement

The homepage introduces the work in three full-screen sections on desktop. Dedicated pages provide more detail.

Homepage

Introduction and operational evidence. The opening section places statistics on the left, the main statement in the center and context on the right. The headline reads, “I build, deploy and operate self-hosted infrastructure and software.” Supporting copy introduces the UniFi network, Proxmox cluster and related services. The landscape sits below and behind the content.

Selected projects. The second section presents three interactive production stacks. A selector, featured project card and facts panel follow the same desktop column structure. The landscape moves toward the top of the viewport as visitors enter this section.

About, writing and contact. The final section, `#continuation`, provides an About excerpt, a selected blog post and contact details. The About excerpt opens with “I started building PCs at 13.” and links to “About Eric.” The blog area combines a category label, date and reading-time metadata, a selected title and an “All blog posts” link. The landscape recedes to the upper right so the text becomes the main focus.

The three type families remain available across this section, but each element uses the role it needs. Contact details do not need a display headline simply to repeat the pattern elsewhere.

Dedicated pages

DESTINATION	PURPOSE
/projects.html	The infrastructure and project archive
/about.html	Personal background, narrative and roadmap
/blog/	Technical articles and incident reviews; 46 articles in the project snapshot

The header contains **Projects**, **About** and **Blog**. These links all lead to pages. Email is placed with contact information lower in the reading sequence, where visitors can choose it after reviewing the work. This also avoids making an email-client action part of the primary page navigation.

Contact

The homepage contact area displays `helloworld@n5hq.me` alongside profile links. GitHub and LinkedIn are listed consistently; the third link remains unresolved between Bluesky and Blog and needs confirmation against the implementation.

Fixed footers on `about.html`, `projects.html` and `blog/` contain email and social links, with contrasting hover treatments.

SECTION 04

Interactive landscape

A procedural pillar field moves from low ripples and terraced peaks to an overhead lattice.

04



SECTION 04

Interactive landscape

The “Faultline” landscape is a procedural Three.js WebGL scene. Low ripples, terraced peaks and an overhead lattice change the appearance of the pillar field as the reader moves through the homepage. Cursor movement and clicks bring smaller, localized changes to the surface. The equations create these visual effects; they are not a validated physical simulation.

Geometry and terrain

The field contains **1,040 pillars**, arranged in **52 columns × 20 rows**. Each pillar is shown through three exposed facets: a left face, a right face and a top cap. The exact cross-section—square or hexagonal—still needs confirmation against the implementation.

Terrain is generated procedurally on the CPU and GPU instead of being loaded as a prebuilt mesh. A `mulberry32` pseudo-random generator supports variation. The summit selection chooses between one and four summits, using thresholds of 0.15, 0.50 and 0.84; the corresponding code appears in the appendix.

Peaks use a terraced height profile with a 3.2 step factor. The profile smooths the transition between terrace levels while keeping the peaks visibly stepped instead of turning them into continuous slopes.

Movement between sections

In the first homepage section, the landscape combines low wave motion with mountain-like peaks at a placement of +35vh.

The transition to the second section is specified as a Y-axis flip involving the camera and scene, with a shift to -32vh. The terrain appears to hang from the top of the viewport. In `#continuation`, it recedes to the upper right as a smaller lattice.

Waves and cursor response

Ambient motion and terrain clicks generate circular wave trains. The wave shader combines a cosine wave with a second harmonic to shape the crest, then applies an asymmetric envelope and distance attenuation. This is a **Stokes-style wave profile**; the excerpt does not show a complete Gerstner displacement model.

The radial attenuation is $1.0 / \sqrt{1.0 + \text{dist} * 0.07}$. This gradually reduces the wave's contribution with distance; it is not inverse-square decay.

For hover interaction, the cursor's two-dimensional position is projected into world space using inverted camera projection matrices. A Gaussian distance weight controls nearby elevation lift, and `damp()` smooths its change over time.

Elevation and color

The shader blends the signal and accent colors according to `vHeight`:

HEIGHT RANGE	COLOR BEHAVIOR
$v\text{Height} < 0.18$	Titan White base color
$0.18 \leq v\text{Height} \leq 0.88$	Smooth transition toward the blue accent
$v\text{Height} \geq 0.88$	Full accent contribution before the brightness multiplier

The multiplier rises from 1.0 to 1.4 as the blend reaches blue. Its purpose is to keep the darker accent visible against the background. The summit therefore uses a brightened blue, not the unmodified CSS value `#002FA7`.

ONE LANDSCAPE / THREE POSITIONS

The terrain follows the reading sequence.

STATE 01

Introduction

Below the opening content; the specified placement is +35vh.



STATE 02

Selected projects

Overhead at the second section; the specified placement is -32vh.



STATE 03

Continuation

A smaller lattice recedes to the upper right.



Figure 02. Three consecutive captures from the live homepage, with the same 1600 × 1000 viewport and framing. Placement values are from the specification. Captured 12 September 2026.

SECTION 05

Responsive behavior and accessibility

Viewport behavior

WIDTH	LAYOUT AND SCROLLING	LANDSCAPE
Desktop, > 820px	Asymmetric three-column layout; full-slide navigation on the homepage and About page	Three.js stage and cursor interaction, with a 2D fallback
Compact, ≤ 820px	Single-column document flow, native scrolling and touch-oriented padding	Static SVG poster, #terrain-poster; WebGL disabled

The compact presentation avoids ongoing WebGL rendering and the desktop terrain scripts, listed at approximately **604 KB**. Actual transfer depends on the delivered assets, compression and cache state.

Desktop slide navigation

Custom wheel handling converts input into section changes. It normalizes `deltaMode` by treating pixel deltas directly, multiplying line deltas by 32, and multiplying page deltas by `window.innerHeight`.

The input threshold is 4px, reduced from 16px. At 100% Windows display scaling, recorded gentle wheel notches produced roughly 10–14px deltas, small enough for the earlier filter to discard. The 4px threshold accepts those inputs. The animation lock was shortened from 750ms to 550ms to reduce the delay between section changes.

ArrowDown, ArrowUp, PageDown, PageUp and Spacebar provide keyboard navigation. Focus guards keep this handling from interfering with text fields and interactive controls. Cross-browser and input-device coverage remains unverified.

RESPONSIVE BEHAVIOR AND ACCESSIBILITY / CONTINUED

Reduced motion

For `prefers-reduced-motion: reduce`, desktop WebGL remains available. The reduced-motion setting disables sinusoidal drift and shockwave ripples, with `config.drift = 0` identified for drift control.

Reduced-motion handling for the entrance curtain, section transitions and cursor attraction remains unresolved. The setting should not be understood to remove all animation.

Layering and readability

The content is placed above the landscape in the stacking order:

LAYER	SELECTOR	Z - INDEX
Header and navigation	<code>.site-header</code>	100
Main content	<code>main, .page</code>	2
WebGL landscape	<code>#terrain-stage</code>	1
2D fallback	<code>#terrain</code>	0

`main` and `.page` use transparent backgrounds, allowing terrain to remain visible behind them. The stacking order puts content in front of the canvas, but does not by itself guarantee sufficient contrast for every text-and-terrain combination. Opacity values and animated backgrounds remain relevant to readability.

SECTION 06

Loading and rendering lifecycle

The desktop entrance uses a curtain of raised pillars that descend into the landscape. Its start is tied to scene initialization, with a timeout intended to release the loading state if initialization does not complete normally.

First-visit sequence

1. The browser requests `index.html`.
2. A script in `<head>` adds `curtain-pending`, showing a dark screen and logo.
3. Preload hints make `three.min.js` and `terrain.js` discoverable early on desktop.
4. During DOM parsing, the 2D canvas `#terrain` remains hidden while `curtain-pending` is active.
5. The dynamically inserted scripts execute in dependency order: `Three.js` first, then the `terrain` script.
6. Successful `createCurtain()` initialization changes the state to `curtain-running`.
7. The curtain settles over `1,600ms`, after which `curtain-running` is removed and the entrance completes.

A `4,000ms` safety timeout replaces the earlier `650ms` timer. The shorter timer could expose the 2D canvas before the `terrain` scripts finished downloading, preventing the intended curtain entrance. Tying the entrance to initialization avoids relying on a fixed download window.

The timeout is intended to release a stalled loading state; it does not guarantee a curtain entrance on every connection. Handling of a late successful initialization after fallback has already started still needs confirmation.

Script delivery

Both scripts have preload hints with `media="(min-width: 821px)"`. Dynamic script elements set `.async = false` before they are appended, preserving the dependency order while allowing the browser to begin fetching both resources without waiting for the first script's `onload` callback.

Rendering fallback

There are three rendering paths:

- **Desktop WebGL:** the full terrain, elevation shader, waves and cursor response.
- **2D Canvas fallback:** the standalone `#terrain` renderer provides a procedural wireframe when the WebGL path fails. It does not depend on Three.js.
- **Compact SVG:** `#terrain-poster` provides a static representation at widths of 820px and below.

Both injected scripts assign `onScriptError` as their error handler. The 2D path is intended to keep a landscape available after script or WebGL failure, provided its own code and assets are available. This is a rendering fallback, not an offline delivery mechanism.

SECTION 07

Editorial approach and publishing

Writing about the work

Portfolio copy should name the systems and describe what happened. “UniFi routing, Proxmox virtualization, Docker containers, and automated storage” tells readers more than a general claim about infrastructure expertise.

Personal context belongs where it explains a decision:

The homelab started as a way out of more than \$200 a month in recurring subscriptions. Running services for other people turned routing, storage, and recovery into disciplined operational work.

Incident writing follows the same approach: describe the failure, the misleading evidence and the lesson. Article titles include:

- “The disk that read 82% free and was 99.9% full”
- “The host wasn’t dead and the backups weren’t real: both passed every check anyway”
- “I diagnosed the same outage twice and got two different answers”

Claims about scale, availability or performance need supporting evidence; when it is missing, describe the observation and its limits.

Blog generation

The blog uses `scripts/build-blog.js`, a Node.js static generator with no third-party dependencies.

The publishing flow is:

```
content/blog/*.md → scripts/build-blog.js → blog/*/index.html
```

The generator produces standalone HTML articles, formats dates, calculates reading-time estimates and extracts opening paragraphs for meta descriptions. The project snapshot contains 46 articles and category filters for **Homelab**, **Projects** and **Web**.

Articles are delivered as crawlable static HTML, with no client-side hydration framework or tracking scripts in the specified architecture.

SECTION 08

Technical reference

Design tokens, component locations and
implementation excerpts.

08



SECTION 08

Technical reference

Design tokens

Tokens in `assets/css/style.css`:

```
:root {  
  /* Color */  
  --ink: #08090c;  
  --ink-mass: #0a0b0e;  
  --signal: #e5e7eb;  
  --signal-72: rgba(229, 231, 235, 0.72);  
  --signal-48: rgba(229, 231, 235, 0.48);  
  --signal-24: rgba(229, 231, 235, 0.24);  
  --accent: #002fa7;  
  
  /* Typography */  
  --kyoto: 'PP Kyoto', Georgia, serif;  
  --authentic: 'AUTHENTIC Sans', system-ui, sans-serif;  
  --pitch: 'Pitch Sans', monospace;  
  
  /* Spacing */  
  --page-x: clamp(24px, 3.8vw, 68px);  
  --edge: clamp(20px, 3.4vw, 56px);  
  --head-space: clamp(96px, 12vh, 140px);  
  
  /* Motion */  
  --transition-fast: 140ms ease;  
  --transition-normal: 180ms ease-out;  
  --transition-snap: 550ms cubic-bezier(0.25, 1, 0.5, 1);  
}
```

Components and implementation locations

IDENTIFIER OR PATH	RESPONSIBILITY
.site-header	Identity and primary navigation
main, .page	Content structure and desktop grid
#continuation	Final homepage section: About, selected writing and contact
#terrain-stage	Three.js rendering layer
#terrain	Standalone 2D Canvas fallback
#terrain-poster	Compact-view SVG illustration
curtain-pending, curtain-running	Entrance lifecycle classes
createCurtain()	Curtain initialization
onScriptError	Script failure handler
assets/js/three.min.js	Three.js dependency
assets/js/terrain.js	Terrain implementation
assets/css/style.css	Shared design tokens and styles
scripts/build-blog.js	Markdown-to-HTML blog generation

SECTION 09

Appendix: implementation excerpts

These excerpts show individual mechanisms, not complete standalone implementations. The identifiers and numeric values match the project specification.

A. Desktop grid

```
.page {  
  display: grid;  
  grid-template-columns:  
    minmax(190px, 23fr)  
    minmax(410px, 49fr)  
    minmax(240px, 28fr);  
  grid-template-rows: repeat(3, minmax(0, 1fr));  
  gap: 18px clamp(24px, 2.5vw, 48px);  
}
```

B. Summit selection and terracing

```
// Weighted selection of one to four summits.  
const roll = random();  
const count = roll < 0.15 ? 1 : roll < 0.50 ? 2 : roll < 0.84 ? 3 : 4;
```

The terrace profile is:

```
steep = clamp((frac(3.2 × dist) - 0.15) / 0.70, 0, 1)  
height = (floor(3.2 × dist) + steep2 × (3 - 2 × steep)) / 3.2
```

Here, `frac` is the fractional part and `clamp` limits the value to the stated range. The definition and units of `dist` in this terrain calculation remain unspecified.

APPENDIX: IMPLEMENTATION EXCERPTS / CONTINUED

C. Wave profile

```
// Cosine profile with a second harmonic to shape the crests.
const float k = 1.396; // Approximately 2 * PI / 4.5 wavelength.
float phase = -dr * k;
float stokes = (cos(phase) + 0.30 * cos(2.0 * phase) + 0.20) * 0.75;

// Narrow leading envelope and longer trailing envelope.
float envelope = (dr > 0.0)
    ? exp(-pow(dr / 1.5, 2.0))
    : exp(-pow(dr / 14.0, 2.0));

// Distance attenuation.
float radialDecay = 1.0 / sqrt(1.0 + dist * 0.07);
height += stokes * envelope * radialDecay * uShockStrength * uShock[i].w;
```

D. Elevation color

```
// Blend from signal to accent as height increases.
float ikbProgress = smoothstep(0.18, 0.88, vHeight);

// Increase brightness toward the summit.
float luminanceBoost = 1.0 + ikbProgress * 0.40;
vec3 finalColor =
    mix(uSignalColor, uAccentColor, ikbProgress) * luminanceBoost;
```

E. Cursor influence

```
W_i = exp(-(d_i / influencePixels)2 × 3.4)
```

The weight decreases with distance from the projected cursor. `damp()` smooths the resulting elevation response.

APPENDIX: IMPLEMENTATION EXCERPTS / CONTINUED

F. Wheel navigation

```
window.addEventListener('wheel', (e) => {
  if (window.innerWidth <= 820) return;
  // deltaMode: 0 = pixels, 1 = lines, 2 = pages.
  const delta = e.deltaMode === 1
    ? e.deltaY * 32
    : e.deltaMode === 2
      ? e.deltaY * window.innerHeight
      : e.deltaY;

  if (Math.abs(delta) < 4) return;
  if (isAnimating) {
    e.preventDefault();
    return;
  }

  if (delta > 0 && currentPageIndex < pages.length - 1) {
    e.preventDefault();
    goToPage(currentPageIndex + 1);
  } else if (delta < 0 && currentPageIndex > 0) {
    e.preventDefault();
    goToPage(currentPageIndex - 1);
  }
}, { passive: false });
```

This excerpt covers wheel input. The keyboard bindings and focus guards described earlier are not shown in it.

APPENDIX: IMPLEMENTATION EXCERPTS / CONTINUED

G. Script preloading and insertion

```
<link rel="preload"
      href="./assets/js/three.min.js"
      as="script"
      media="(min-width: 821px)">
<link rel="preload"
      href="./assets/js/terrain.js"
      as="script"
      media="(min-width: 821px)">
```

```
// Allow fetching without an onload chain; retain dependency order.
const s1 = document.createElement('script');
s1.src = "./assets/js/three.min.js";
s1.async = false;
s1.onerror = onScriptError;

const s2 = document.createElement('script');
s2.src = "./assets/js/terrain.js";
s2.async = false;
s2.onerror = onScriptError;

document.head.append(s1, s2);
```

PRODUCTION NOTES

Source & visual record

Document basis

DESIGN-SPECIFICATION.md, supplied for this edition. The document's technical descriptions, code excerpts and open implementation questions are retained. Authoring-only figure comments have been replaced by the two implementation figures.

Implementation visuals

Figures 01 and 02 were captured from natchannnn.github.io/eri-cli/ on 12 September 2026. The cover and section openers use a crop of the same live terrain capture. The screenshots show the deployed site on that date; they do not establish a performance benchmark or accessibility audit.

Typography & layout

PP Kyoto and AUTHENTIC Sans carry the display and body roles. Consolas provides the monospaced metadata role in this PDF. The website's Void Black, Titan White and International Klein Blue palette is adapted with lighter reading pages. The 23 : 49 : 28 grid acts as a recurring compositional motif.

Reading this edition

The main text explains design intent and behavior. The technical reference and appendix preserve identifiers, numerical values and implementation excerpts for closer inspection. Source files accompany this PDF.